

ARB SPOTTER

Code Explanation & Project Documentation

Arb Spotting — The Foundations

Arbitrage betting is a guaranteed, risk-free profit strategy. By placing wagers on all possible outcomes of a single event, it is possible to exploit price discrepancies across different bookmakers. Bookmakers often offer different odds to differentiate themselves, which can create situations where placing each side of a bet guarantees a profit regardless of the outcome.

The formula used to detect arbs is:

$$\text{Implied Probability} = (1 / \text{Decimal Odds}) \times 100$$

Example Scenario:

Bookmaker A — Player X to win: 2.20 → $1 / 2.20 \times 100 = 45.45\%$

Bookmaker B — Player Y to win: 2.00 → $1 / 2.00 \times 100 = 50.00\%$

Total implied probability = $45.45\% + 50.00\% = 95.45\%$

Since $95.45\% < 100\%$, an arbitrage exists. Profit margin = 4.55% .

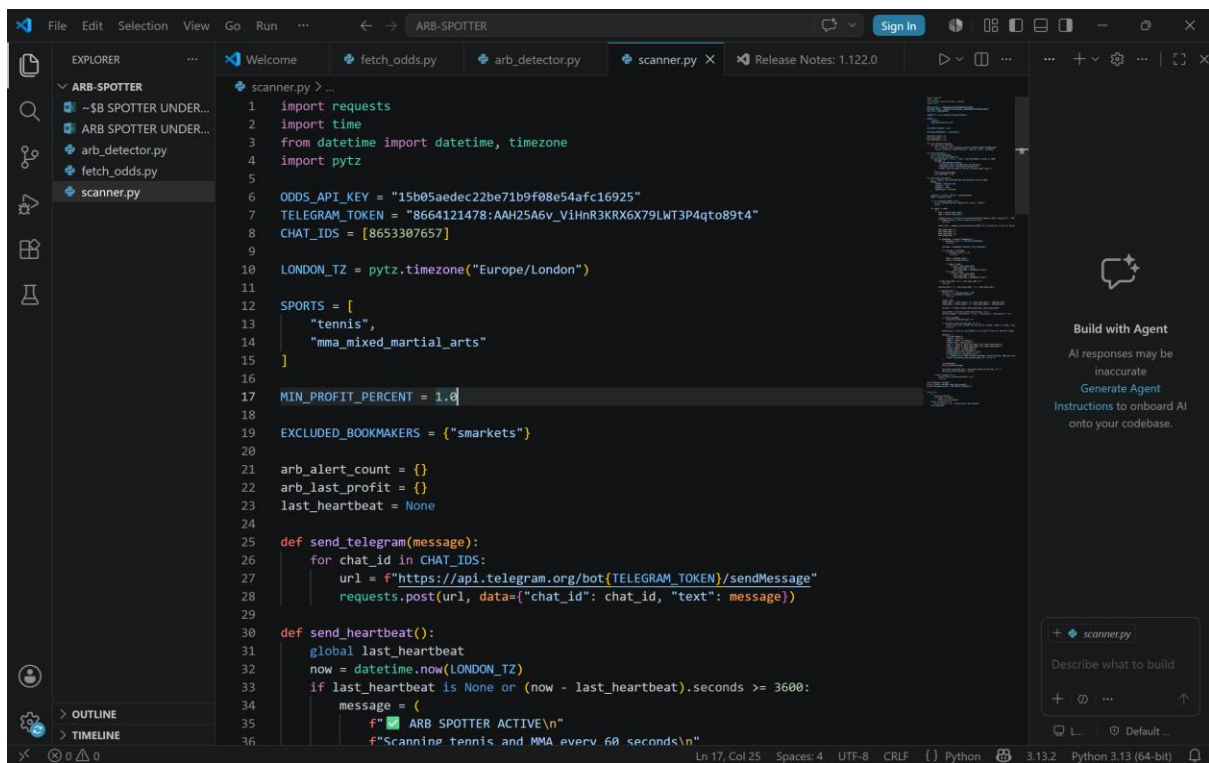
Decision Making in Our Project

Tennis was chosen as the target market for two reasons: matches are played continuously, providing a constant stream of opportunities, and outcomes are binary (win/loss), meaning the arbitrage condition reduces to a single check — whether the sum of the two best implied probabilities across all bookmakers falls below one. Football's three-way market would require placing three simultaneous legs, complicating both the mathematics and execution.

Odds data is sourced from The Odds API, which aggregates live prices from around 20 UK bookmakers in a single call, removing the need to scrape each bookmaker individually. This comes at a cost, so the operational objective is for any arbs exploited to at least cover the API subscription.

The scanner runs 24/7 on a DigitalOcean cloud droplet rather than locally, ensuring consistent uptime without depending on a personal machine being on and connected. A 60-second polling interval is used to spread API calls and avoid exhausting the monthly call limit prematurely — the tradeoff being that arbs detected near the end of a polling cycle may have already closed by the time an alert is acted on. When an arb is found, the details are pushed automatically via a Telegram bot, allowing immediate notification regardless of location.

Code Walkthrough



```
1 import requests
2 import time
3 from datetime import datetime, timezone
4 import pytz
5
6 ODDS_API_KEY = "15b05aedec22be71b9f08e54afc16925"
7 TELEGRAM_TOKEN = "8664121478:AAH25A6v_ViHnR3KRX6X79LWT3P4qto89t4"
8 CHAT_IDS = [8653307657]
9
10 LONDON_TZ = pytz.timezone("Europe/London")
11
12 SPORTS = [
13     "tennis",
14     "mma_mixed_martial_arts"
15 ]
16
17 MIN_PROFIT_PERCENT = 1.0
18
19 EXCLUDED_BOOKMAKERS = {"smarkets"}
20
21 arb_alert_count = {}
22 arb_last_profit = {}
23 last_heartbeat = None
24
25 def send_telegram(message):
26     for chat_id in CHAT_IDS:
27         url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendMessage"
28         requests.post(url, data={"chat_id": chat_id, "text": message})
29
30 def send_heartbeat():
31     global last_heartbeat
32     now = datetime.now(LONDON_TZ)
33     if last_heartbeat is None or (now - last_heartbeat).seconds >= 3600:
34         message = (
35             f"ARB SPOTTER ACTIVE\n"
36             f"Scanning tennis and MMA every 60 seconds\n"
```

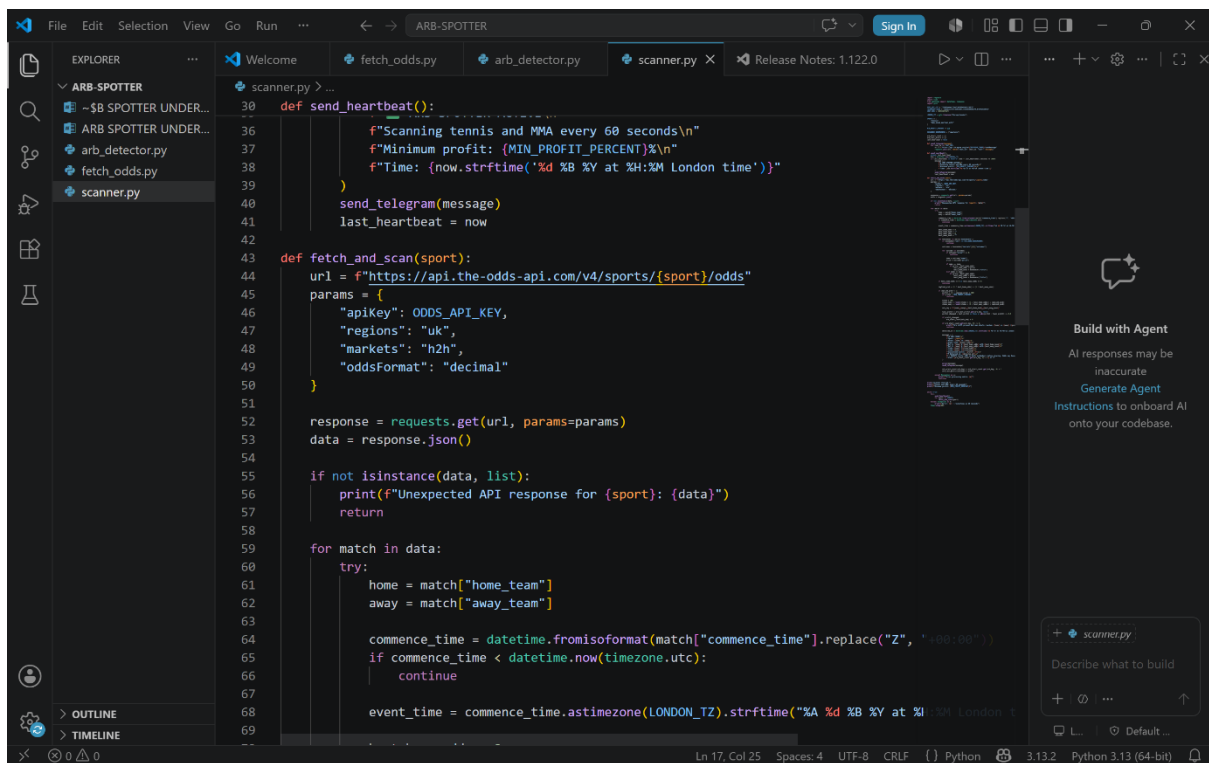
Lines 1–4 Import all required libraries.

Lines 6–8 Define constants — API key, Telegram token, and configuration variables.

Lines 10+ Set the time zone using pytz, storing it in LONDON_TZ.

Lines 25–28 Define the Telegram function. Builds the API URL using an f-string and loops through every chat_id in the list, sending to all recipients not just one.

Lines 30–41 Heartbeat function — checks whether more than 3600 seconds have elapsed since the last heartbeat and sends a confirmation message if so. "Last heartbeat is None" handles the first run: the condition is true so the first heartbeat fires immediately. The global keyword allows the function to update the last_heartbeat variable that lives outside its scope, so the next iteration has something to compare against.

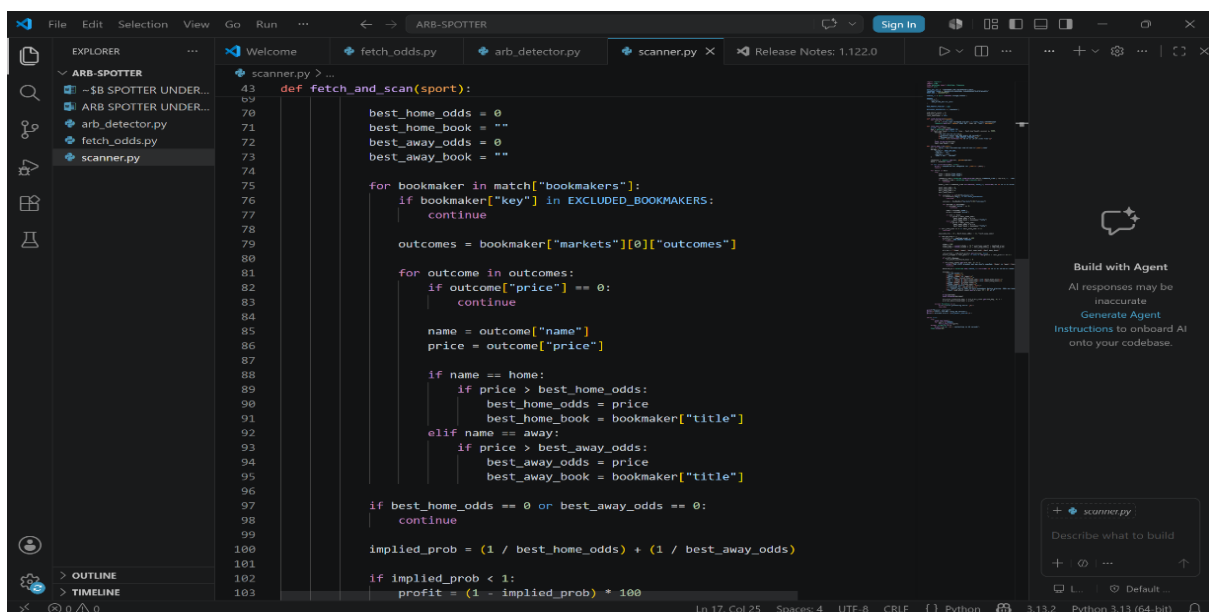


```
30 def send_heartbeat():
31     f"Scanning tennis and MMA every 60 seconds\n"
32     f"Minimum profit: {MIN_PROFIT_PERCENT}%\n"
33     f"Time: {now.strftime('%d %B %Y at %H:%M London time')}"
34 )
35 send_telegram(message)
36 last_heartbeat = now
37
38 def fetch_and_scan(sport):
39     url = f"https://api.the-odds-api.com/v4/sports/{sport}/odds"
40     params = {
41         "apiKey": ODDS_API_KEY,
42         "regions": "uk",
43         "markets": "h2h",
44         "oddsFormat": "decimal"
45     }
46
47     response = requests.get(url, params=params)
48     data = response.json()
49
50     if not isinstance(data, list):
51         print(f"Unexpected API response for {sport}: {data}")
52         return
53
54     for match in data:
55         try:
56             home = match["home_team"]
57             away = match["away_team"]
58
59             commence_time = datetime.fromisoformat(match["commence_time"].replace("Z", "+00:00"))
60             if commence_time < datetime.now(timezone.utc):
61                 continue
62
63             event_time = commence_time.astimezone(LONDON_TZ).strftime("%A %d %B %Y at %H:%M London time")
```

Lines 43–50 Build the API parameters dictionary. regions, markets, apikey, and odds Format are fixed values defined by the API documentation. UK, h2h, and decimal are predefined — this tells the API to return only UK-region, head-to-head, decimal-format odds.

Lines 55–57 Validate the API response format. If the response is not what was expected, an "unexpected API response" message is printed.

Lines 59–66 Filter out in-play matches. If commence_time < datetime.now() the event has already started, so the code uses continue to skip it and move to the next match.



```
43 def fetch_and_scan(sport):
44     best_home_odds = 0
45     best_home_book = ""
46     best_away_odds = 0
47     best_away_book = ""
48
49     for bookmaker in match["bookmakers"]:
50         if bookmaker["key"] in EXCLUDED_BOOKMAKERS:
51             continue
52
53         outcomes = bookmaker["markets"][0]["outcomes"]
54
55         for outcome in outcomes:
56             if outcome["price"] == 0:
57                 continue
58
59             name = outcome["name"]
60             price = outcome["price"]
61
62             if name == home:
63                 if price > best_home_odds:
64                     best_home_odds = price
65                     best_home_book = bookmaker["title"]
66             elif name == away:
67                 if price > best_away_odds:
68                     best_away_odds = price
69                     best_away_book = bookmaker["title"]
70
71     if best_home_odds == 0 or best_away_odds == 0:
72         continue
73
74     implied_prob = (1 / best_home_odds) + (1 / best_away_odds)
75
76     if implied_prob < 1:
77         profit = (1 - implied_prob) * 100
```

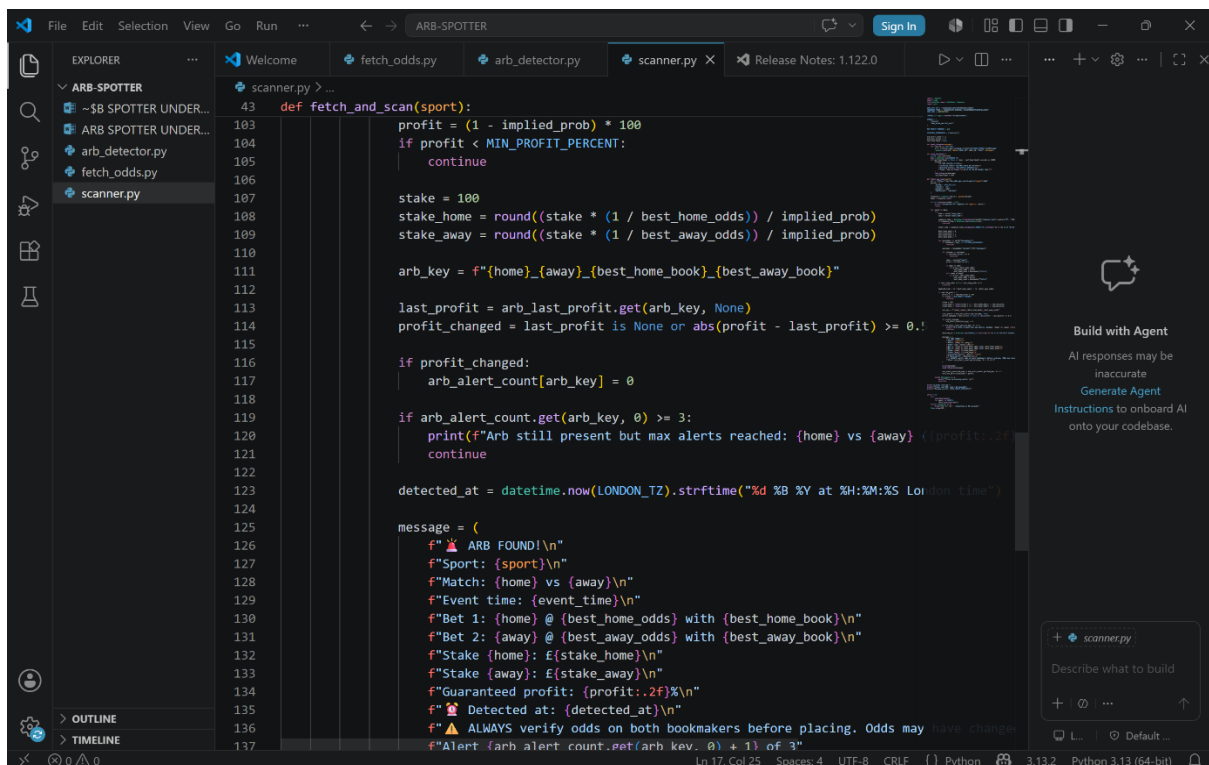
Lines 70–73 Initialise tracker variables before the loop. Best home and away odds are set to 0, and the corresponding bookmaker names to empty strings. These act as a starting point that gets updated as you iterate through bookmakers.

Lines 75–77 Skip excluded bookmakers. If the bookmaker key is in the exclusion list, continue moves on to the next bookmaker.

Lines 80–95 Find the best available odds across all bookmakers for each outcome. If a price is higher than the current best, it becomes the new best. If either the best home or away odds remain at 0 (missing or bad data), the match is skipped.

`["markets"][0]["outcomes"]` navigates into the API response structure: markets list → first market → outcomes array.

Lines 97–99 Calculate implied probabilities for each side using the best available odds.



```
def fetch_and_scan(sport):
    profit = (1 - implied_prob) * 100
    if profit < MIN_PROFIT_PERCENT:
        continue

    stake = 100
    stake_home = round((stake * (1 / best_home_odds)) / implied_prob)
    stake_away = round((stake * (1 / best_away_odds)) / implied_prob)

    arb_key = f"{home}_{away}_{best_home_book}_{best_away_book}"

    last_profit = arb_last_profit.get(arb_key, None)
    profit_changed = last_profit is None or abs(profit - last_profit) >= 0.5

    if profit_changed:
        arb_alert_count[arb_key] = 0

    if arb_alert_count.get(arb_key, 0) >= 3:
        print(f"Arb still present but max alerts reached: {home} vs {away} (profit: {profit:.2f})")
        continue

    detected_at = datetime.now(LONDON_TZ).strftime("%d %B %Y at %H:%M:%S London time")

    message = (
        f"🚀 ARB FOUND!\n"
        f"Sport: {sport}\n"
        f"Match: {home} vs {away}\n"
        f"Event time: {event_time}\n"
        f"Bet 1: {home} @ {best_home_odds} with {best_home_book}\n"
        f"Bet 2: {away} @ {best_away_odds} with {best_away_book}\n"
        f"Stake {home}: £{stake_home}\n"
        f"Stake {away}: £{stake_away}\n"
        f"Guaranteed profit: {profit:.2f}%\n"
        f"🕒 Detected at: {detected_at}\n"
        f"⚠️ ALWAYS verify odds on both bookmakers before placing. Odds may have changed!\n"
        f"Alert {arb_alert_count.get(arb_key, 0) + 1} of 3"
```

Lines 103–105 Profit threshold filter — if the profit margin is below 1%, the opportunity is ignored as not worth executing.

Lines 107–114 Calculate stake allocation. Stakes are split proportionally by implied probability across a fixed total stake, guaranteeing equal return on either outcome. The code also detects whether the profit has shifted by more than 0.5% since the last alert and resets the alert counter if so.

Lines 116–121 Alert deduplication logic. The same arb will trigger a maximum of 3 alerts. If the profit changes materially the counter resets; if the count reaches 3 the code logs "arb still present but max alerts reached" and moves on.

Line 123 Record the timestamp of when the arb was detected.

Lines 125–137 Construct the Telegram message string, inserting all relevant variables.

```
135 def fetch_and_scan(sport):
136     f" 🚨 Detected at: {detected_at}\n"
137     f" ⚠️ ALWAYS verify odds on both bookmakers before placing. Odds may
138     f"Alert {arb_alert_count.get(arb_key, 0) + 1} of 3"
139 )
140 print(message)
141 send_telegram(message)
142
143 arb_alert_count[arb_key] = arb_alert_count.get(arb_key, 0) + 1
144 arb_last_profit[arb_key] = profit
145
146 except Exception as e:
147     print(f"Error processing match: {e}")
148     continue
149
150 print("Scanner started.")
151 print(f"Tennis and MMA: every 60 seconds")
152 print(f"Minimum profit: {MIN_PROFIT_PERCENT}%")
153
154
155 while True:
156     try:
157         send_heartbeat()
158         for sport in SPORTS:
159             fetch_and_scan(sport)
160     except Exception as e:
161         print(f"Error: {e} - restarting in 60 seconds")
162         time.sleep(60)
```

Lines 140–141 Execute the `send_telegram` function with the constructed message.

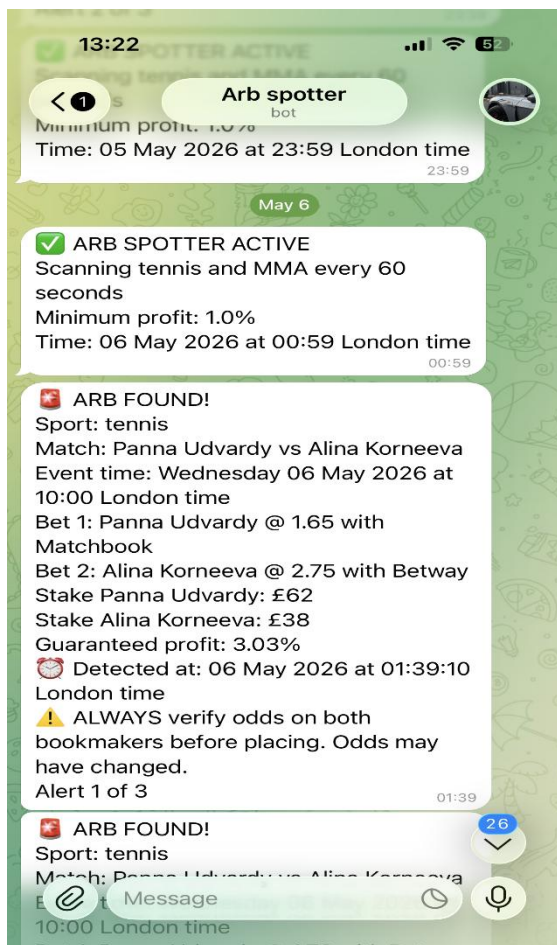
Lines 143–144 Update the alert count and last-recorded profit for use in the next iteration.

Lines 146–148 Except block — catches any errors during match processing (e.g. missing API data), prints the error, and uses `continue` to move on rather than crashing the whole program.

Lines 150–152 Print statements confirming the scanner is active.

Lines 155–162 Main loop. `while True` runs forever. Calls `send_heartbeat()` then iterates through each sport calling `fetch_and_scan()`. If anything crashes, the `except` block catches it and `time.sleep(60)` pauses for 60 seconds before retrying.

Executed Telegram Signal



Limitations of Our Arb Spotter

Although our scanner functions correctly — it collects data from the API, computes the combined implied probability across bookmakers, and pushes a notification to Telegram for us to act on — it is not practically possible to take advantage of most opportunities. By the time an alert is read and two legs are placed by hand, the favourable price has usually moved. Manual execution is not viable; near-simultaneous automated placement would be required to resolve the speed constraint. But even that only addresses latency — the more fundamental limitation is that bookmakers identify and restrict accounts that consistently arb.

The Odds API does not provide real-time tick data, and the 60-second polling loop adds further lag. These differences of seconds are critical in liquid markets. The strategy also demands significant committed capital: even at zero risk, balances must sit idle across all bookmakers simultaneously. With profit margins of approximately 1–5% per trade, a £500 balance yields only £5–£25 per trade. Across ten bookmakers that is £5,000 of committed capital — generating returns that are modest relative to the capital tied up, the account attrition risk, and the operational overhead involved.

6-Step Blueprint

1. Import libraries
2. Define constant / config variables

3. Define your functions
4. Filter logic — keep moving if satisfied
5. Execute the output (send message)
6. Main loop to keep it running

This is a standard structure for any automated monitoring script — the skeleton stays the same regardless of the specific use case.

Libraries Used

requests Makes HTTP calls to the Odds API and Telegram API.

time Used for `time.sleep(60)` to pause between scan cycles.

datetime / timezone Handles and formats times correctly throughout the script.

pytz Handles named time zone conversion (Europe/London) — the built-in `datetime` module does not support named time zones.

Python Concepts Used

def Defines a reusable function so you don't repeat code.

while True Runs the program forever until manually stopped.

for loops Iterates through bookmakers, sports, and outcomes.

if / elif / else Decision making at each filter stage.

continue Skips the current iteration and moves to the next one.

try / except Catches errors so the program doesn't crash.

global Allows a function to modify a variable that lives outside it.

f-strings Builds readable strings with variables inserted inline.

isinstance() Checks if data is the type you expect.

dictionary.get() Safely retrieves a value without crashing if the key doesn't exist.

round() Rounds stake calculations to clean numbers.

abs() Returns the absolute value.